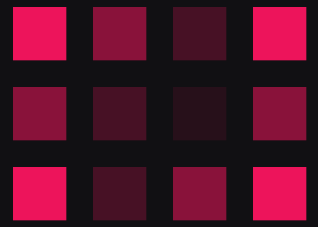
DOMAIN DRIVEN DESIGN · LIVE · CAPÍTULO 12

Banco de dados com JDBC

Do Maven ao CRUD completo: a sua coleção de games salva no Oracle.



Prof. Me. Thiago T. I. Yamamoto | FIAP

AGENDA

O que vamos construir hoje

01
Projeto
Maven e o driver Oracle

02
Conexão
JDBC e ConnectionFactory

03
Tabela
tb_game e a sequence

04
Model
A classe Game

05
DAO
O CRUD, método a método

06
View
O menu no console

No caminho: 5 quizzes no chat. No final: um exercício com gabarito.

POR QUE UM BANCO DE DADOS

Fechou o programa, perdeu o save

Memória RAM

Os objetos que você cria com **new** vivem na RAM. Ela é volátil: o programa terminou, os dados sumiram.

Banco de dados

Guarda os dados de forma organizada e permanente. Dá para consultar, gerar relatórios e alterar depois.

JDBC

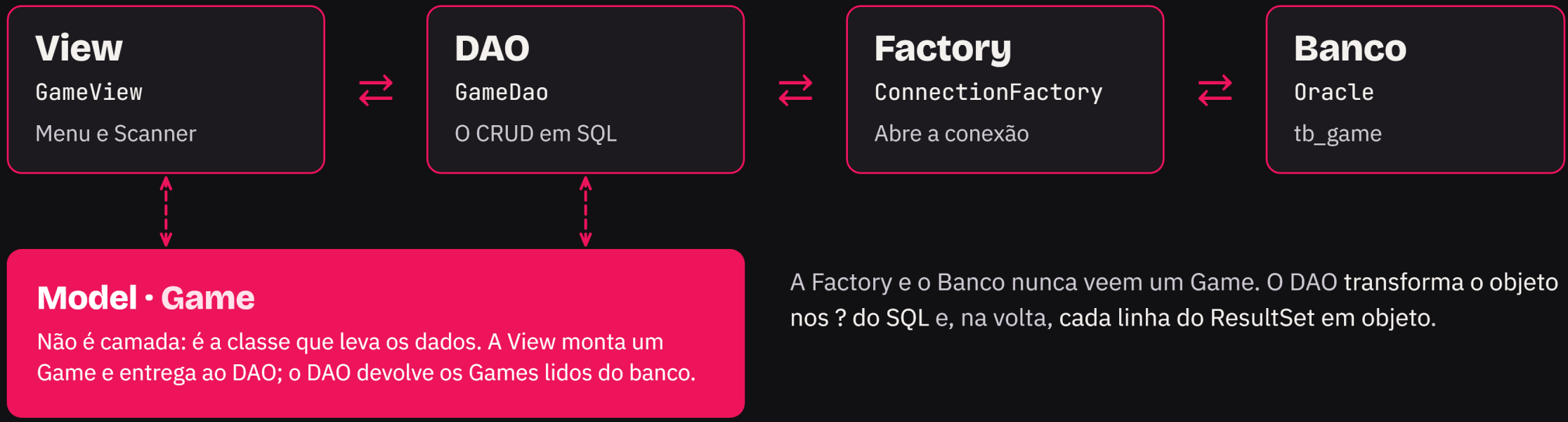
Java DataBase Connectivity: a biblioteca que liga o seu código Java ao banco relacional.

Pense num jogo sem save: você joga 40 horas, fecha e volta para a fase 1. É isso que acontece com um sistema sem banco.

O MAPA DA LIVE

Quatro camadas e um objeto que viaja entre elas

Cada camada só conversa com a vizinha. Vamos construir de trás para frente: primeiro o banco e a conexão, por último o menu.



A Factory e o Banco nunca veem um Game. O DAO transforma o objeto nos ? do SQL e, na volta, cada linha do ResultSet em objeto.

01

Preparando o projeto

Maven cria a estrutura e baixa o driver do Oracle para você.

FIAP

MAVEN

Um arquivo descreve o projeto inteiro

- ✓ **Automatiza** compilação, testes e empacotamento do projeto Java
- ✓ **Gerencia dependências**: você declara a biblioteca e ele baixa o .jar, com a versão certa
- ✓ Tudo fica no **pom.xml**, o mesmo em qualquer máquina da turma

Sem Maven, o driver do Oracle seria um .jar baixado do site e adicionado à mão em cada computador. Com Maven, são 5 linhas no pom.xml.

Tag do pom.xml	Para quê
groupId	A organização: br.com.fiap
artifactId	O nome do projeto
version	A versão do projeto
packaging	O tipo do pacote: jar
properties	Configurações, como o encoding
dependencies	As bibliotecas que o projeto usa

PASSO A PASSO · INTELLIJ IDEA

Criando o projeto Maven

01 **File › New › Project** e, à esquerda, escolha **Maven Archetype**

02 Name: **live-jdbc**. JDK: **17** ou mais recente

03 Catalog: **Internal**. Archetype: **maven-archetype-quickstart**

04 Em **Advanced Settings**, preencha o GroupId e o ArtifactId

05 Clique em **Create** e espere o IntelliJ baixar o que precisa

Campo	Valor
Name	live-jdbc
JDK	17
Catalog	Internal
Archetype	maven-archetype-quickstart
GroupId	br.com.fiap
ArtifactId	live-jdbc
Version	1.0-SNAPSHOT

A ESTRUTURA

Onde fica cada coisa

Projeto

```
live-jdbc
├── pom.xml
└── src
    └── main
        └── java
            └── br.com.fiap
                ├── factory
                │   └── ConnectionFactory
                ├── model
                │   └── Game
                ├── dao
                │   └── GameDao
                ├── exception
                │   └── EntidadeNaoEncontradaException
                └── view
                    └── GameView
```

Pasta ou arquivo	Guarda
pom.xml	Configuração e dependências
src/main/java	O código do projeto
src/test/java	Testes (não usaremos hoje)

- ✓ Crie os 5 pacotes antes de começar: **factory**, **model**, **dao**, **exception** e **view**
- ✓ Um pacote por responsabilidade: **view**, **dao** e **factory** são as camadas; **model** e **exception** são as classes que circulam entre elas

PASSO A PASSO · O DRIVER

A dependência do Oracle no pom.xml

pom.xml · dentro de <project>

```xml
<dependencies>
    <dependency>
        <groupId>com.oracle.database.jdbc</groupId>
        <artifactId>ojdbc11</artifactId>
        <version>21.9.0.0</version>
    </dependency>
</dependencies>
```

01 Cole a dependência dentro de **<dependencies>**

02 Salve e clique no ícone do Maven que aparece no canto do editor para recarregar

03 Confira em **External Libraries**: o ojdbc11 tem que estar lá

>_ Sem o driver no projeto

```
java.sql.SQLException:
  No suitable driver found for
  jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl
```

- ✓ O erro mais comum da primeira aula: o código compila, mas não há driver para o Oracle
- ✓ Desde o JDBC 4, não precisa de Class.forName: o driver se registra sozinho

QUAL DRIVER USAR

ojdbc8 ou ojdbc11?

artifactId	Java suportado	Especificação	Quando usar
ojdbc8	Java 8 e 11	JDBC 4.2	Projetos antigos, em Java 8
ojdbc11	Java 11, 17 e 21	JDBC 4.3	O nosso caso: Java 17 ou mais recente

groupId

Sempre com.oracle.database.jdbc

version

Pegue a mais recente no mvnrepository.com, buscando por ojdbc11

Material do capítulo

O PDF usa o ojdbc8. Com Java 17, troque só o artifactId para ojdbc11

QUIZ 1 · RESPONDA NO CHAT

O que o Maven faz quando você adiciona o ojdbc11 no pom.xml?

A Cria a tabela no banco de dados automaticamente

B Baixa o .jar do driver e coloca no projeto

C Conecta o IntelliJ direto no Oracle da FIAP

D Converte o código Java em SQL

QUIZ 1 · RESPOSTA

O que o Maven faz quando você adiciona o ojdbc11 no pom.xml?

A Cria a tabela no banco de dados automaticamente

B Baixa o .jar do driver e coloca no projeto

C Conecta o IntelliJ direto no Oracle da FIAP

D Converte o código Java em SQL

Resposta B. O Maven só gerencia bibliotecas: baixa o driver e o adiciona ao classpath. Conexão e tabela continuam sendo com a gente.

02

Conectando no banco

Uma API do Java, um driver para cada banco.

FIAP

O mesmo código para qualquer banco

O JDBC define interfaces. Cada fabricante entrega um **driver** que implementa essas interfaces. Trocar Oracle por MySQL troca o driver, não o seu código.



É a **#09** na prática: você programa pela interface, o driver é a implementação.

FIAP

A STRING DE CONEXÃO

O endereço do banco

```
jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl
```

Parte	O que é	No nosso caso
jdbc:oracle:thin	O protocolo e o tipo de driver	Sempre igual no Oracle
@HOST	O servidor do banco	oracle.fiap.com.br
PORTA	A porta do serviço	1521
SID	O nome da instância	orcl

✓ Além da string, o **DriverManager** pede o usuário e a senha: o seu RM e a senha do Oracle da FIAP

FIAP

PASSO A PASSO · PRIMEIRO TESTE

Conectou? Então roda!

App.java · dentro do main

```
try {
    Connection conexao = DriverManager.getConnection(
            "jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl",
"rm00000", "000000");
    System.out.println("Conexão realizada!");
    conexao.close();
} catch (SQLException e) {
    System.out.println(e.getMessage());
}
```

01 Imports do pacote **java.sql**: Connection, DriverManager e SQLException

02 Troque rm00000 e 000000 pelo seu usuário e senha

>_ Deu certo

```
Conexão realizada!
```

>_ Usuário ou senha errados

```
ORA-01017: invalid username/password;
           logon denied
```

- ✓ **SQLException** é checked: o compilador obriga o try-catch
- ✓ Fora da rede da FIAP? Pode precisar da VPN

FIAP

PADRÃO FACTORY

ConnectionFactory: a conexão num lugar só

ConnectionFactory.java · pacote factory

```java
public class ConnectionFactory {

    private static final String URL = "jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl";
    private static final String USUARIO = "rm00000";
    private static final String SENHA = "000000";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, SENHA);
    }
```

- ✓ Sem a fábrica, cada classe repetiria a URL, o usuário e a senha
- ✓ Mudou a senha? Você altera **uma** linha
- ✓ static: chama direto pela classe, sem new. ConnectionFactory.getConnection()
- ✓ final: constantes, ninguém muda o valor

Factory é um padrão de projeto: esconde como o objeto é criado e entrega pronto para quem pede.

FIAP

QUIZ 2 · RESPONDA NO CHAT

Por que o método getConnection() da ConnectionFactory é static?

A Para a conexão nunca fechar

B Para chamar sem criar um objeto da fábrica

C Porque o Oracle exige métodos static

D Para deixar a conexão mais rápida

FIAP

QUIZ 2 · RESPOSTA

Por que o método getConnection() da ConnectionFactory é static?

A Para a conexão nunca fechar

B Para chamar sem criar um objeto da fábrica

C Porque o Oracle exige métodos static

D Para deixar a conexão mais rápida

Resposta B. Com static, o método pertence à classe: ConnectionFactory.getConnection(), sem new ConnectionFactory().

03

A tabela

Antes do Java, o banco precisa saber o que guardar.

FIAP

PASSO A PASSO · SQL DEVELOPER

tb_game e a sua sequence

script.sql

```
CREATE SEQUENCE sq_tb_game
    START WITH 1
    INCREMENT BY 1
    NOCACHE;

CREATE TABLE tb_game (
    cd_game NUMBER(10) PRIMARY KEY,
    nm_game VARCHAR2(80) NOT NULL,
    nr_nota NUMBER(3,1),
    nr_ano NUMBER(4),
    dt_compra DATE,
    st_finalizado NUMBER(1)
);
```

Prefixo	Significa
cd_	Código, a chave
nm_	Nome
nr_	Número
dt_	Data
st_	Status: 1 sim, 0 não

- ✓ A **sequence** gera o próximo código: sq_tb_game.NEXTVAL
- ✓ Oracle não tem boolean em tabela: **NUMBER(1)** com 0 ou 1

FIAP

TESTE ANTES DO JAVA

Se funciona no SQL Developer, funciona no JDBC

SQL Developer

```
INSERT INTO tb_game (cd_game, nm_game, nr_nota, nr_ano, dt_compra, st_finalizado)
VALUES (sq_tb_game.NEXTVAL, 'Hades', 9.3, 2020, DATE '2026-02-14', 1);

SELECT * FROM tb_game;
```

CD_GAME	NM_GAME	NR_NOTA	NR_ANO	DT_COMPRA	ST_FINALIZADO
1	Hades	9,3	2020	14/02/26	1

- ✓ O JDBC só leva esse mesmo SQL para o banco. Errou no SQL Developer, vai errar no Java
- ✓ No SQL Developer, confirme com **COMMIT**. O JDBC confirma sozinho (auto-commit)

04

O model

Uma classe Java com a cara de uma linha da tabela.

FIAP

PASSO A PASSO · PACOTE MODEL

A classe Game

Game.java

```java
public class Game {

    private Long codigo;
    private String nome;
    private double nota;
    private int ano;
    private LocalDate dataCompra;
    private boolean finalizado;

    public Game() {
    }

    public Game(String nome, double nota, int ano, LocalDate dataCompra, boolean finalizado) {
        this.nome = nome;
        this.nota = nota;
        this.ano = ano;
        this.dataCompra = dataCompra;
        this.finalizado = finalizado;
    }

    public Game(Long codigo, String nome, double nota, int ano, LocalDate dataCompra, boolean finalizado) {
        this(nome, nota, ano, dataCompra, finalizado);
        this.codigo = codigo;

    // getters, setters e toString
}
```

01 Um atributo **private** para cada coluna da tabela

02 Construtor **sem o código**: para cadastrar, quem gera o código é a sequence

03 Construtor **com o código**: para montar o objeto que veio do banco

04 Getters e setters: **Alt + Insert** no IntelliJ gera para você

✓ this(...) reaproveita o outro construtor, como na #06

MAPEAMENTO

Coluna, atributo e o método do JDBC

Coluna	Tipo Oracle	Atributo	Tipo Java	Grava com	Lê com
cd_game	NUMBER(10)	codigo	Long	(sequence)	getLong
nm_game	VARCHAR2(80)	nome	String	setString	getString
nr_nota	NUMBER(3,1)	nota	double	setDouble	getDouble
nr_ano	NUMBER(4)	ano	int	setInt	getInt
dt_compra	DATE	dataCompra	LocalDate	setDate	getDate
st_finalizado	NUMBER(1)	finalizado	boolean	setInt	getInt

`LocalDate ↔ java.sql.Date`

```
stm.setDate(4, Date.valueOf(game.getDataCompra()));
rs.getDate("dt_compra").toLocalDate();
```

`boolean ↔ NUMBER(1)`

```
stm.setInt(5, game.isFinalizado() ? 1 : 0);
rs.getInt("st_finalizado") == 1;
```

QUIZ 3 · RESPONDA NO CHAT

No model, por que o construtor usado para cadastrar não recebe o código?

A Porque código é sempre zero

B Porque a sequence do banco gera o código no INSERT

C Porque o Java não aceita Long em construtor

D Porque o código é gerado pelo Scanner

FIAP

QUIZ 3 · RESPOSTA

No model, por que o construtor usado para cadastrar não recebe o código?

A Porque código é sempre zero

B Porque a sequence do banco gera o código no INSERT

C Porque o Java não aceita Long em construtor

D Porque o código é gerado pelo Scanner

Resposta B. O INSERT usa sq_tb_game.NEXTVAL: quem cria o código é o banco. O Java só conhece o código quando lê o registro de volta.

FIAP

05

O DAO

Todo o SQL do sistema numa classe só, um método por operação.

FIAP

PADRÃO DAO · DATA ACCESS OBJECT

O esqueleto primeiro

GameDao.java · pacote dao

```java
public class GameDao {

    private Connection conexao;

    public GameDao() throws SQLException {
        conexao = ConnectionFactory.getConnection();
    }

    public void cadastrar(Game game) { }
    public Game pesquisar(long codigo) { return null; }
    public List<Game> listar() { return null; }
    public void atualizar(Game game) { }
    public void remover(long codigo) { }

    public void fecharConexao() throws SQLException {
        conexao.close();
    }
}
```

- ✓ O DAO é o **intermediário** entre a aplicação e o banco. Só ele escreve SQL
- ✓ A conexão abre no **construtor** e fica num atributo: todos os métodos usam a mesma
- ✓ fecharConexao() fecha quando o sistema termina
- ✓ Um DAO por tabela: GameDao, SerieDao, ClienteDao...

Os métodos ainda estão vazios. Nos próximos slides, preenchemos um por vez, do C ao D do CRUD.

CADASTRAR · PRIMEIRA TENTATIVA

cadastrar | pesquisar | listar | atualizar | remover

Funciona. E está errado.

GameDao.java · versão com Statement

```
public void cadastrar(Game game) throws SQLException {
    Statement stm = conexao.createStatement();
    stm.executeUpdate("INSERT INTO tb_game (cd_game, nm_game, nr_nota) "
            + "VALUES (sq_tb_game.NEXTVAL, '" + game.getNome() + "', "
            + game.getNota() + ")");
}
```

- ⚠ Monta o SQL **concatenando** Strings com os dados do usuário
- ⚠ Precisa lembrar das aspas simples em cada texto
- ⚠ O valor digitado vira **parte do comando**: a porta para o SQL injection

cadastrar | pesquisar | listar | atualizar | remover

E se o usuário digitar um pedaço de SQL?

>_ Busca por nome: o usuário digita

```
x' OR '1'='1
```

Statement concatena e monta

```
SELECT nm_game FROM tb_game
WHERE nm_game = 'x' OR '1' = '1'
```

>_ SqlInjectionView

```
Statement encontrou 4: [God of War, Hades, ...]
PreparedStatement encontrou 0: []
```

O ataque

O apóstrofo fecha o texto e o **OR '1'='1'** é sempre verdadeiro: a busca devolve a tabela inteira. Num login, é entrar sem senha.

A defesa

PreparedStatement: o valor vai separado do comando. O banco procura um jogo chamado literalmente x' OR '1'='1.

FIAP

PASSO A PASSO · C DO CRUD

cadastrar | pesquisar | listar | atualizar | remover

cadastrar com PreparedStatement

GameDao.java

```
public void cadastrar(Game game) throws SQLException {
    String sql = "INSERT INTO tb_game (cd_game, nm_game, nr_nota, nr_ano, dt_compra, "
+ "st_finalizado) VALUES (sq_tb_game.NEXTVAL, ?, ?, ?, ?, ?)";
    PreparedStatement stm = conexao.prepareStatement(sql);
    stm.setString(1, game.getNome());
    stm.setDouble(2, game.getNota());
    stm.setInt(3, game.getAno());
    stm.setDate(4, Date.valueOf(game.getDataCompra()));
    stm.setInt(5, game.isFinalizado() ? 1 : 0);
    stm.executeUpdate();
    stm.close();
}
```

01 Um ? no lugar de cada valor. O código fica com a sequence

02 Preencha cada ? com o **set** do tipo certo. A contagem começa em **1**

03 executeUpdate() roda INSERT, UPDATE ou DELETE

04 close() libera o PreparedStatement

QUIZ 4 · RESPONDA NO CHAT

Qual é a principal vantagem do PreparedStatement sobre o Statement?

A Dispensa a conexão com o banco

B Protege contra SQL injection, separando dado de comando

C Cria a tabela quando ela não existe

D Permite escrever o SQL sem WHERE

QUIZ 4 · RESPOSTA

Qual é a principal vantagem do PreparedStatement sobre o Statement?

A Dispensa a conexão com o banco

B Protege contra SQL injection, separando dado de comando

C Cria a tabela quando ela não existe

D Permite escrever o SQL sem WHERE

Resposta B. Os valores entram pelos set, não pela concatenação. O banco nunca interpreta o que o usuário digitou como SQL.

FIAP

PASSO A PASSO · R DO CRUD

✓ cadastrar | pesquisar | listar | atualizar | remover

pesquisar: SELECT com ResultSet

GameDao.java

```
public Game pesquisar(long codigo) throws SQLException, EntidadeNaoEncontradaException {
    PreparedStatement stm = conexao.prepareStatement("SELECT * FROM tb_game WHERE "
+ "cd_game = ?");
    stm.setLong(1, codigo);
    ResultSet rs = stm.executeQuery();
    if (!rs.next()) {
        throw new EntidadeNaoEncontradaException("Game " + codigo + " não encontrado");
    }
    Game game = extrairGame(rs);
    stm.close();
    return game;
}
```

01 executeQuery() roda o SELECT e devolve um **ResultSet**

02 O ResultSet é um cursor: começa **antes** da primeira linha

03 next() avança e devolve **false** se não tem linha: aí lançamos a exceção

04 extrairGame(rs) transforma a linha num objeto Game

MÉTODO AUXILIAR

✓ cadastrar | pesquisar | listar | atualizar | remover

extrairGame: a linha vira objeto

GameDao.java · private

```
private Game extrairGame(ResultSet rs) throws SQLException {
    Long codigo = rs.getLong("cd_game");
    String nome = rs.getString("nm_game");
    double nota = rs.getDouble("nr_nota");
    int ano = rs.getInt("nr_ano");
    LocalDate data = rs.getDate("dt_compra").toLocalDate();
    boolean finalizado = rs.getInt("st_finalizado") == 1;
    return new Game(codigo, nome, nota, ano, data, finalizado);
}
```

- ✓ Os **get** leem a coluna pelo nome, na linha atual do cursor
- ✓ **private**: só o DAO usa. Pesquisar e listar chamam o mesmo método
- ✓ Sem ele, o mapeamento coluna → atributo ficaria copiado em todo SELECT

FIAP

PACOTE EXCEPTION

✓ cadastrar | pesquisar | listar | atualizar | remover

E quando o código não existe?

EntidadeNaoEncontradaException.java

```java
package br.com.fiap.exception;

public class EntidadeNaoEncontradaException extends Exception {

    public EntidadeNaoEncontradaException(String mensagem) {
        super(mensagem);
    }
}
```

- ✓ Uma exceção **nossa**, checked, porque herda de **Exception**
- ✓ Nome genérico de propósito: serve para o GameDao, o SerieDao, qualquer DAO
- ✓ O DAO **lança**. Quem decide a mensagem para o usuário é a View

Nada de System.out dentro do DAO: ele avisa com exceções, a tela conversa com o usuário.

PASSO A PASSO · R DO CRUD

✓ cadastrar ✓ pesquisar **listar** atualizar remover

listar: um while no ResultSet

GameDao.java

```
public List<Game> listar() throws SQLException {
    PreparedStatement stm = conexao.prepareStatement("SELECT * FROM tb_game
 ORDER BY cd_game");
    ResultSet rs = stm.executeQuery();
    List<Game> lista = new ArrayList<>();
    while (rs.next()) {
        lista.add(extrairGame(rs));
    }
    stm.close();
    return lista;
}
```

- ✓ Mesmo esqueleto do pesquisar, mas com while no lugar do if
- ✓ Cada volta do while = uma linha da tabela = um Game na lista
- ✓ Tabela vazia? A lista volta vazia, sem exceção

É a #11 encontrando o banco: o resultado de uma consulta chega como uma List de objetos.

FIAP

PASSO A PASSO · U DO CRUD

✓ cadastrar | ✓ pesquisar | ✓ listar | atualizar | remover

atualizar: UPDATE com WHERE

GameDao.java

```java
public void atualizar(Game game) throws SQLException, EntidadeNaoEncontradaException {
    String sql = "UPDATE tb_game SET nm_game = ?, nr_nota = ?, nr_ano = ?, "
            + "dt_compra = ?, st_finalizado = ? WHERE cd_game = ?";
    PreparedStatement stm = conexao.prepareStatement(sql);
    stm.setString(1, game.getNome());
    stm.setDouble(2, game.getNota());
    stm.setInt(3, game.getAno());
    stm.setDate(4, Date.valueOf(game.getDataCompra()));
    stm.setInt(5, game.isFinalizado() ? 1 : 0);
    stm.setLong(6, game.getCodigo());
    int linhas = stm.executeUpdate();
    stm.close();
    if (linhas == 0) {
        throw new EntidadeNaoEncontradaException("Game " + game.getCodigo() + " não encontrado");
    }
}
```

- ✓ O código entra no WHERE: é o último ?, o de número **6**
- ✓ executeUpdate() devolve quantas linhas mudaram
- ✓ 0 linhas? O código não existe: lançamos a exceção

UPDATE sem WHERE altera a tabela inteira. Confira sempre.

PASSO A PASSO · D DO CRUD

✓ cadastrar ✓ pesquisar ✓ listar ✓ atualizar remover

remover: DELETE pela chave

GameDao.java

```
public void remover(long codigo) throws SQLException, EntidadeNaoEncontradaException {
    PreparedStatement stm = conexao.prepareStatement("DELETE FROM tb_game WHERE"
+  cd_game = ?");
    stm.setLong(1, codigo);
    int linhas = stm.executeUpdate();
    stm.close();
    if (linhas == 0) {
        throw new EntidadeNaoEncontradaException("Game " + codigo + " não encontrado");
    }
}
```

- ✓ Mesma lógica do atualizar: executeUpdate e conferir as linhas
- ✓ DELETE de um código que não existe **não dá erro** no banco. Por isso a checagem
- ✓ Com isso, o GameDao está completo: C, R, U e D

FIAP

QUIZ 5 · RESPONDA NO CHAT

O que acontece se o while (rs.next()) do listar virar um if?

A Dá erro de compilação

B Lista todos os jogos do mesmo jeito

C Devolve no máximo um jogo: o primeiro da consulta

D Apaga os outros registros da tabela

FIAP

QUIZ 5 · RESPOSTA

O que acontece se o while (rs.next()) do listar virar um if?

A Dá erro de compilação

B Lista todos os jogos do mesmo jeito

C Devolve no máximo um jogo: o primeiro da consulta

D Apaga os outros registros da tabela

Resposta C. O if lê só a primeira linha do cursor. É o que o pesquisar faz de propósito, já que busca por um único código.

06

A view

O menu que junta tudo: o usuário escolhe, o DAO executa.

FIAP

PASSO A PASSO · PACOTE VIEW

GameView: o menu

GameView.java · dentro do main

```
Scanner scanner = new Scanner(System.in);
try {
    GameDao dao = new GameDao();
    int opcao;
    do {
        System.out.println("═══ MINHA COLEÇÃO DE GAMES ═══");
        System.out.println("1-Cadastrar 2-Pesquisar 3-Listar 4-Atualizar 5-Remover 0-Sair");
        opcao = scanner.nextInt();
        scanner.nextLine();
        switch (opcao) {
            case 1 → cadastrar(scanner, dao);
            case 2 → pesquisar(scanner, dao);
            case 3 → listar(dao);
            case 4 → atualizar(scanner, dao);
            case 5 → remover(scanner, dao);
            case 0 → System.out.println("Até a próxima partida!");
            default → System.out.println("Opção inválida");
        }
    } while (opcao ≠ 0);
    dao.fecharConexao();
} catch (SQLException e) {
    System.out.println("Erro no banco: " + e.getMessage());
```

01 Cria o DAO uma vez: a conexão abre aqui

02 **do-while** repete o menu até digitar 0

03 **switch** chama um método por opção

04 Saiu do laço: fecha a conexão

✓ A View trata a **SQLException**, o DAO só propaga

FIAP

SCANNER SEM SUSTOS

Lendo um Game do teclado

GameView.java

```java
private static Game lerGame(Scanner scanner) {
    System.out.print("Nome: ");
    String nome = scanner.next() + scanner.nextLine();
    System.out.print("Nota: ");
    double nota = scanner.nextDouble();
    scanner.nextLine();
    System.out.print("Ano de lançamento: ");
    int ano = scanner.nextInt();
    scanner.nextLine();
    System.out.print("Data da compra (aaaa-mm-dd): ");
    LocalDate data = LocalDate.parse(scanner.next());
    System.out.print("Já zerou? (s/n): ");
    boolean finalizado = scanner.next().equalsIgnoreCase("s");
    scanner.nextLine();
    return new Game(nome, nota, ano, data, finalizado);
}
```

Leitura	Faça assim
Texto com espaço	next() + nextLine()
Número	nextInt() e depois nextLine()
Data	LocalDate.parse(next())

- ✓ God of War tem espaços: só o next() leria **God**
- ✓ O nextLine() depois do número limpa o Enter que ficou no buffer
- ✓ Com o Java em português, a nota se digita com **vírgula**: 9,5

FIAP

RODANDO

A coleção salva no Oracle

```
>_ Cadastrar

═══ MINHA COLEÇÃO DE GAMES ═══
1-Cadastrar 2-Pesquisar 3-Listar 4-Atualizar 5-Remover 0-Sair
1
Nome: God of War
Nota: 9,5
Ano de lançamento: 2018
Data da compra (aaaa-mm-dd): 2026-03-10
Já zerou? (s/n): s
Game cadastrado!
```

```
>_ Listar, pesquisar e sair

3
1 | God of War | nota 9.5 | 2018 | comprado em 2026-03-10 | zerado
2 | Hollow Knight | nota 9.0 | 2017 | comprado em 2026-05-02 | jogando
2 game(s) na coleção
2
Código: 7
Game 7 não encontrado
0
Até a próxima partida!
```

Feche o programa e rode de novo: a opção 3 continua mostrando os jogos. É o save que faltava lá no começo.

FIAP

SE FOR LER UM SLIDE SÓ, LEIA ESTE

Resumo da live

Maven
ojdbc11 no pom.xml

Factory
A conexão num lugar só

Tabela
tb_game e sq_tb_game

Model
Game: uma linha vira objeto

DAO
PreparedStatement e ResultSet

View
Menu e Scanner, sem SQL

A ordem de construção: pom.xml → ConnectionFactory → tabela → model → exception → DAO → view.

07

Exercício

Agora é a sua vez: outro domínio, o mesmo caminho.

FIAP

EXERCÍCIO · MARATONA

O catálogo de séries

Você vai criar, do zero, um sistema para controlar as séries que está assistindo. Mesmo roteiro da live, outra tabela.

Coluna	Tipo	Regra
cd_serie	NUMBER(10)	Chave, pela sequence sq_tb_serie
nm_serie	VARCHAR2(80)	Obrigatório
ds_plataforma	VARCHAR2(40)	Obrigatório
nr_temporadas	NUMBER(3)	
dt_estreia	DATE	
st_concluida	NUMBER(1)	1 sim, 0 não

O que entregar

script.sql com a tabela e a sequence

Serie, SerieDao, ConnectionFactory e a exceção

SerieView com o menu completo

Tente sem olhar o GameDao. Travou? Volte só no slide do método que travou.

EXERCÍCIO · REQUISITOS

Passo a passo

01 Projeto Maven **maratona-jdbc**, com o ojdbc11 no pom.xml

02 Script com **tb_serie** e **sq_tb_serie**. Rode no SQL Developer

03 **Serie** com os 6 atributos e dois construtores: com e sem o código

04 **SerieDao** com cadastrar, pesquisar, listar (ordenado pelo nome), atualizar e remover, todos com PreparedStatement

05 Método extra: **buscarPorPlataforma(String)**, que acha netflix, Netflix ou NETFLIX

06 pesquisar, atualizar e remover lançam **EntidadeNaoEncontradaException** quando o código não existe

07 **SerieView** com as opções de 1 a 6 e 0 para sair. Nenhum SQL na View

Pista para o passo 5: UPPER(ds_plataforma) LIKE ? e o valor com % dos dois lados.

FIAP

EXERCÍCIO · COMO DEVE FUNCIONAR

Execução esperada

>_ SerieView · depois de cadastrar 3 séries

```
1-Cadastrar 2-Pesquisar 3-Listar 4-Por plataforma 5-Atualizar 6-Remover 0-Sair
3
3 | Dark | Netflix | 3 temp. | estreia 2017-12-01 | concluída
1 | Stranger Things | Netflix | 5 temp. | estreia 2016-07-15 | concluída
2 | The Last of Us | Max | 2 temp. | estreia 2023-01-15 | em andamento
3 série(s)
4
Plataforma: netflix
3 | Dark | Netflix | 3 temp. | estreia 2017-12-01 | concluída
1 | Stranger Things | Netflix | 5 temp. | estreia 2016-07-15 | concluída
2 série(s)
6
Código: 9
Série 9 não encontrada
```

- ✓ A lista sai em **ordem alfabética**: é o ORDER BY no SQL, não no Java
- ✓ A busca ignora maiúsculas: digitou netflix, achou Netflix
- ✓ Código inexistente mostra a mensagem da exceção, sem derrubar o programa

08

Gabarito

Confira depois de tentar. Errar antes de ver a resposta faz parte.

FIAP

GABARITO · PASSOS 2 E 3

script.sql

```sql
CREATE SEQUENCE sq_tb_serie
    START WITH 1
    INCREMENT BY 1
    NOCACHE;

CREATE TABLE tb_serie (
    cd_serie NUMBER(10) PRIMARY KEY,
    nm_serie VARCHAR2(80) NOT NULL,
    ds_plataforma VARCHAR2(40) NOT NULL,
    nr_temporadas NUMBER(3),
    dt_estreia DATE,
    st_concluida NUMBER(1)
);
```

Serie.java

```java
public class Serie {

    private Long codigo;
    private String nome;
    private String plataforma;
    private int temporadas;
    private LocalDate estreia;
    private boolean concluida;

    public Serie(String nome, String plataforma, int temporadas, LocalDate estreia, boolean concluida) {
        this.nome = nome;
        this.plataforma = plataforma;
        this.temporadas = temporadas;
        this.estreia = estreia;
        this.concluida = concluida;
    }

    public Serie(Long codigo, String nome, String plataforma, int temporadas,
                 LocalDate estreia, boolean concluida) {
        this(nome, plataforma, temporadas, estreia, concluida);
        this.codigo = codigo;

    // getters, setCodigo e toString
}
```


FIAP

GABARITO · SERIEDAO · PARTE 1

SerieDao.java

```java
public class SerieDao {

    private Connection conexao;

    public SerieDao() throws SQLException {
        conexao = ConnectionFactory.getConnection();
    }

    public void cadastrar(Serie serie) throws SQLException {
        String sql = "INSERT INTO tb_serie (cd_serie, nm_serie, ds_plataforma, nr_temporadas, "
                + "dt_estreia, st_concluida) VALUES (sq_tb_serie.NEXTVAL, ?, ?, ?, ?, ?)";
        PreparedStatement stm = conexao.prepareStatement(sql);
        stm.setString(1, serie.getNome());
        stm.setString(2, serie.getPlataforma());
        stm.setInt(3, serie.getTemporadas());
        stm.setDate(4, Date.valueOf(serie.getEstreia()));
        stm.setInt(5, serie.isConcluida() ? 1 : 0);
        stm.executeUpdate();
        stm.close();
    // continua…
```

- ✓ A conexão abre no construtor, igual ao GameDao
- ✓ 5 ? para 5 valores: o código fica com sq_tb_serie.NEXTVAL
- ✓ st_concluida grava 1 ou 0 com o operador ternário

FIAP

GABARITO · SERIEDAO · PESQUISAR

SerieDao.java

```java
public Serie pesquisar(long codigo) throws SQLException, EntidadeNaoEncontradaException {
    PreparedStatement stm = conexao.prepareStatement("SELECT * FROM tb_serie WHERE cd_serie = ?");
    stm.setLong(1, codigo);
    ResultSet rs = stm.executeQuery();
    if (!rs.next()) {
        throw new EntidadeNaoEncontradaException("Série " + codigo + " não encontrada");
    }
    Serie serie = extrairSerie(rs);
    stm.close();
    return serie;
}
```

SerieDao.java · private

```java
private Serie extrairSerie(ResultSet rs) throws SQLException {
    Long codigo = rs.getLong("cd_serie");
    String nome = rs.getString("nm_serie");
    String plataforma = rs.getString("ds_plataforma");
    int temporadas = rs.getInt("nr_temporadas");
    LocalDate estreia = rs.getDate("dt_estreia").toLocalDate();
    boolean concluida = rs.getInt("st_concluida") == 1;
    return new Serie(codigo, nome, plataforma, temporadas, estreia, concluida);
}
```

- ✓ if (!rs.next()): nenhuma linha, lança a exceção
- ✓ extrairSerie é reaproveitado por pesquisar, listar e buscarPorPlataforma

FIAP

GABARITO · SERIEDAO · PARTE 2

SerieDao.java

```java
public List<Serie> listar() throws SQLException {
    PreparedStatement stm = conexao.prepareStatement("SELECT * FROM tb_serie ORDER BY nm_serie");
    return executarLista(stm);
}
```

o método extra

```java
public List<Serie> buscarPorPlataforma(String plataforma) throws SQLException {
    PreparedStatement stm = conexao.prepareStatement(
            "SELECT * FROM tb_serie WHERE UPPER(ds_plataforma) LIKE ? ORDER BY nm_serie");
    stm.setString(1, "%" + plataforma.toUpperCase() + "%");
    return executarLista(stm);
}
```

private: listar e buscar usam

```java
private List<Serie> executarLista(PreparedStatement stm) throws SQLException {
    ResultSet rs = stm.executeQuery();
    List<Serie> lista = new ArrayList<>();
    while (rs.next()) {
        lista.add(extrairSerie(rs));
    }
    stm.close();
    return lista;
}
```

- ✓ listar e buscarPorPlataforma só mudam o SQL. O resto foi para **executarLista**
- ✓ "%" + plataforma.toUpperCase() + "%": o % vai no valor, não no SQL
- ✓ O ? continua protegendo contra injection, mesmo com LIKE

FIAP

GABARITO · SERIEDAO · PARTE 3

SerieDao.java

```java
public void atualizar(Serie serie) throws SQLException, EntidadeNaoEncontradaException {
    String sql = "UPDATE tb_serie SET nm_serie = ?, ds_plataforma = ?, nr_temporadas = ?, "
            + "dt_estreia = ?, st_concluida = ? WHERE cd_serie = ?";
    PreparedStatement stm = conexao.prepareStatement(sql);
    stm.setString(1, serie.getNome());
    stm.setString(2, serie.getPlataforma());
    stm.setInt(3, serie.getTemporadas());
    stm.setDate(4, Date.valueOf(serie.getEstreia()));
    stm.setInt(5, serie.isConcluida() ? 1 : 0);
    stm.setLong(6, serie.getCodigo());
    int linhas = stm.executeUpdate();
    stm.close();
    if (linhas == 0) {
        throw new EntidadeNaoEncontradaException("Série " + serie.getCodigo() + " não encontrada");
    }
}
```

SerieDao.java

```java
public void remover(long codigo) throws SQLException, EntidadeNaoEncontradaException {
    PreparedStatement stm = conexao.prepareStatement("DELETE FROM tb_serie WHERE cd_serie = ?");
    stm.setLong(1, codigo);
    int linhas = stm.executeUpdate();
    stm.close();
    if (linhas == 0) {
        throw new EntidadeNaoEncontradaException("Série " + codigo + " não encontrada");
    }
}
```

- ✓ 0 linhas afetadas = código inexistente = exceção
- ✓ No atualizar, o código é o último ?: o de número 6
- ✓ fecharConexao() igual ao do GameDao

FIAP

GABARITO · SERIEVIEW · O MENU

SerieView.java · dentro do try do main

```
SerieDao dao = new SerieDao();
int opcao;
do {
    System.out.println("═══ MARATONA ═══");
    System.out.println("1-Cadastrar 2-Pesquisar 3-Listar 4-Por plataforma 5-Atualizar 6-Remover 0-Sair");
    opcao = scanner.nextInt();
    scanner.nextLine();
    try {
        switch (opcao) {
            case 1 → {
                dao.cadastrar(lerSerie(scanner));
                System.out.println("Série cadastrada!");
            }
            case 2 → System.out.println(dao.pesquisar(lerCodigo(scanner)));
            case 3 → exibir(dao.listar());
            case 4 → {
                System.out.print("Plataforma: ");
                String plataforma = scanner.next() + scanner.nextLine();
                exibir(dao.buscarPorPlataforma(plataforma));
            }
            case 5 → {
                long codigo = lerCodigo(scanner);
                Serie serie = lerSerie(scanner);
                serie.setCodigo(codigo);
                dao.atualizar(serie);
                System.out.println("Série atualizada!");
            }
            case 6 → {
                dao.remover(lerCodigo(scanner));
                System.out.println("Série removida!");
            }
            case 0 → System.out.println("Fim da maratona!");
            default → System.out.println("Opção inválida");
        }
    } catch (EntidadeNaoEncontradaException e) {
        System.out.println(e.getMessage());
    }
} while (opcao ≠ 0);
```

- ✓ Um único try-catch para a **EntidadeNaoEncontradaException**, dentro do laço: o erro aparece e o menu continua
- ✓ A **SQLException** fica no try de fora, como na GameView
- ✓ case com seta: sem break, sem cair no próximo

FIAP

GABARITO · SERIEVIEW · LEITURAS

SerieView.java

```java
private static Serie lerSerie(Scanner scanner) {
    System.out.print("Nome: ");
    String nome = scanner.next() + scanner.nextLine();
    System.out.print("Plataforma: ");
    String plataforma = scanner.next() + scanner.nextLine();
    System.out.print("Temporadas: ");
    int temporadas = scanner.nextInt();
    scanner.nextLine();
    System.out.print("Estreia (aaaa-mm-dd): ");
    LocalDate estreia = LocalDate.parse(scanner.next());
    System.out.print("Concluída? (s/n): ");
    boolean concluida = scanner.next().equalsIgnoreCase("s");
    scanner.nextLine();
    return new Serie(nome, plataforma, temporadas, estreia, concluida);
}
```

SerieView.java

```java
private static long lerCodigo(Scanner scanner) {
    System.out.print("Código: ");
    long codigo = scanner.nextLong();
    scanner.nextLine();
    return codigo;
}
```

SerieView.java

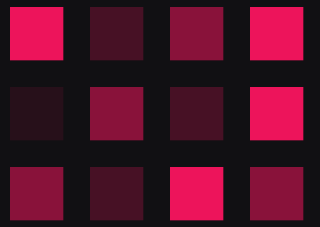
```java
private static void exibir(List<Serie> series) {
    for (Serie s : series) {
        System.out.println(s);
    }
    System.out.println(series.size() + " série(s)");
}
```

✓ Nome e plataforma podem ter espaço: next() + nextLine()

FIAP

“Os dados são algo precioso e vão durar mais do que os próprios sistemas.”

Tim Berners-Lee



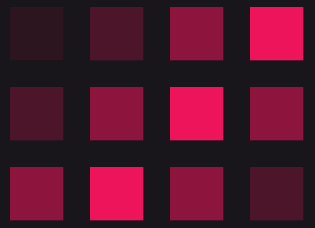
FIAP

DIREITOS AUTORAIS

Copyright © 2020 – 2027

Prof. Me. Thiago T. I. Yamamoto

Todos os direitos reservados. A reprodução ou divulgação total ou parcial deste documento é expressamente proibida sem o consentimento formal, por escrito, do professor (autor).



FIAP